# Sixth Assignment: Computational Neuroscience

Pierina Milla

October 2026

**Methods.** All programs are written in Fortran 90 (Appendix) and use units of ms, mV, pA, pF, MΩ and nS, for which $1\,\mathrm{pA/pF} = 1\,\mathrm{mV/ms}$ and $1\,\mathrm{nS}\cdot\mathrm{mV} = 1\,\mathrm{pA}$. The deterministic runs use the Euler method and the noisy runs the Euler–Maruyama method, with $\Delta t = 0.01$ ms and $t_{\max} = 2$ s unless stated otherwise. Spike times are obtained by linear interpolation, $t^* = t_{k-1} + \Delta t\,(V_L - V_{k-1})/(V_k - V_{k-1})$, and the absolute refractory period holds $V = V_{\mathrm{redef}}$ until $t^* + \tau_{\mathrm{ref}}$. Firing rates are $f = N_{\mathrm{spikes}}/t_{\max}$ (resolution 0.5 Hz). Every noisy run uses a different random seed.

# 1 Question 1: Leaky integrate-and-fire neuron

The model is $C\dot{V} = (V_{\mathrm{rep}} - V)/R + I$ with $V_{\mathrm{rep}} = -70$ mV, $R = 40$ MΩ, $C = 250$ pF, $V_L = -50$ mV, $V_{\mathrm{redef}} = -65$ mV, $\tau_{\mathrm{ref}} = 2$ ms. The membrane time constant is $\tau_m = RC = 10$ ms.

## 1.1 (a) $f$–$I$ curve

**Analytical result.** For constant $I$ the solution is $V(t) = V_\infty + (V_{\mathrm{redef}} - V_\infty)e^{-t/\tau_m}$ with $V_\infty = V_{\mathrm{rep}} + RI$. The threshold is reached only if $V_\infty > V_L$, i.e. $I > I_L = (V_L - V_{\mathrm{rep}})/R = 500$ pA. The time to go from $V_{\mathrm{redef}}$ to $V_L$ is $\tau_m \ln \frac{RI-(V_{\mathrm{redef}}-V_{\mathrm{rep}})}{RI-(V_L-V_{\mathrm{rep}})}$, so

$$f(I) = \left[\tau_{\mathrm{ref}} + \tau_m \ln \frac{RI - (V_{\mathrm{redef}} - V_{\mathrm{rep}})}{RI - (V_L - V_{\mathrm{rep}})}\right]^{-1} \quad (I > I_L), \qquad f = 0 \ (I \le I_L). \tag{1}$$

Setting $f = 150$ Hz gives $RI = 62.0$ mV, i.e. $I \approx 1551$ pA, so the sweep covers $400 \le I \le 1600$ pA in steps of 10 pA.

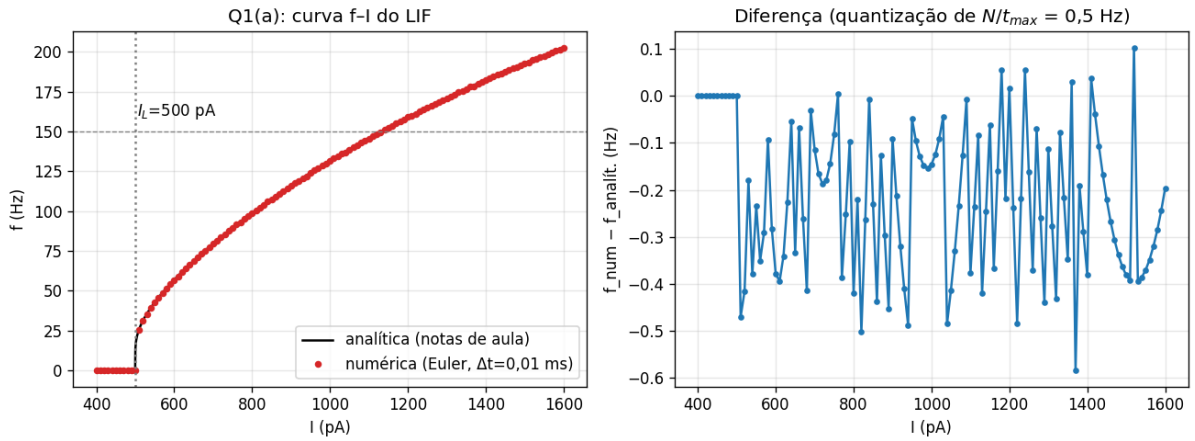



Figure 1: LIF $f$–$I$ curve: numerical (symbols) and analytical (line). Right: difference.

The numerical curve agrees with the formula to within 0.58 Hz (maximum), which is the 0.5 Hz quantization of $N/t_{\max}$. The curve is zero up to $I_L = 500$ pA and then rises continuously from zero frequency (logarithmic divergence of the period at $I_L$), a type-I behavior.

# 2 Question 2: LIF neuron with noise

The noisy model is $\dot{V} = (V_{\text{rep}} - V)/\tau_m + I/C + \sigma w(t)$, integrated with $V_{n+1} = V_n + f(V_n)\Delta t + \sigma\tilde{w}_n\sqrt{\Delta t}$, $\tilde{w}_n \sim \mathcal{N}(0,1)$ (Box–Muller). For subthreshold $I$ the free membrane potential is an Ornstein–Uhlenbeck process with mean $V_\infty$ and stationary standard deviation $\sigma_V = \sigma\sqrt{\tau_m/2}$ (= 1.12 mV for $\sigma = 0.5$ and 3.35 mV for $\sigma = 1.5$).

## 2.1 (a) $f$–$I$ curves with noise

Increasing $\sigma$ from zero, the curve first departs from the $\sigma = 0$ one at $\sigma_1 = 0.5$ mV/ms$^{1/2}$ (non-zero rates appear just below $I_L$, e.g. $f(450\,\text{pA}) = 13$ Hz and $f(500\,\text{pA}) = 29.5$ Hz versus 0 Hz) and it changes markedly again at $\sigma_2 = 1.5$ mV/ms$^{1/2}$ (firing already at 400 pA, $f = 18.5$ Hz, and $f(500) = 39.5$ Hz).

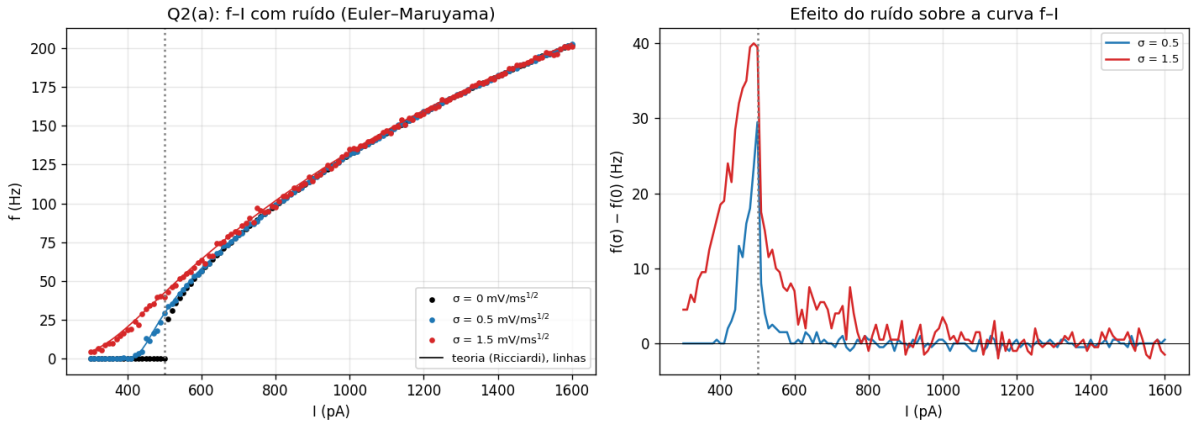



Figure 2: $f$–$I$ for $\sigma = 0$, 0.5 and 1.5 mV/ms$^{1/2}$. Lines on the left panel: Siegert–Ricciardi theory, Eq. (2).

**Interpretation.** Below $I_L$ the mean drive cannot reach threshold ($V_\infty < V_L$) but fluctuations can: spikes are noise-driven and the sharp corner of the deterministic curve at $I_L$ is smoothed into a sigmoid-like foot; the larger $\sigma$, the further below $I_L$ the firing extends and the more the threshold is blurred (maximum change $\approx 30$ Hz at $I_L$ for $\sigma_1$, $\approx 40$ Hz for $\sigma_2$). Far above $I_L$ the neuron is mean-driven, the interspike interval is dominated by the deterministic charging time, and the curves practically coincide (differences are within the $\pm\sqrt{N}/t_{\max} \approx 3$–$5$ Hz sampling error of a 2-s run). The theoretical rate of the LIF with white noise (Siegert–Ricciardi) is

$$f^{-1} = \tau_{\text{ref}} + \tau_m\sqrt{\pi}\int_{(V_{\text{redef}}-V_\infty)/\sigma_B}^{(V_L-V_\infty)/\sigma_B} e^{u^2}\,[1+\operatorname{erf}(u)]\,du, \qquad \sigma_B = \sigma\sqrt{\tau_m}, \tag{2}$$

and reproduces the simulations (maximum difference 3.6 and 5.3 Hz for $\sigma_1, \sigma_2$, again of the order of the sampling error); an independent Monte Carlo first-passage calculation at $I = 450$ pA, $\sigma = 1.5$ gives 31.2 Hz, in agreement with Eq. (2) (31.2 Hz) and with the simulation (31.4 Hz with $\Delta t = 5 \times 10^{-4}$ ms).

## 2.2 (b) Effect of $\Delta t$ on the subthreshold trajectory

$I = 300$ pA ($V_\infty = -58$ mV, below threshold), $\sigma = 1.5$ mV/ms$^{1/2}$, 20 ms. A single Brownian path is generated at the finest step ($10^{-5}$ ms) and summed to produce the increments for $\Delta t = 10^{-2}, 10^{-3}, 10^{-4}$ ms, so that the trajectories are directly comparable.

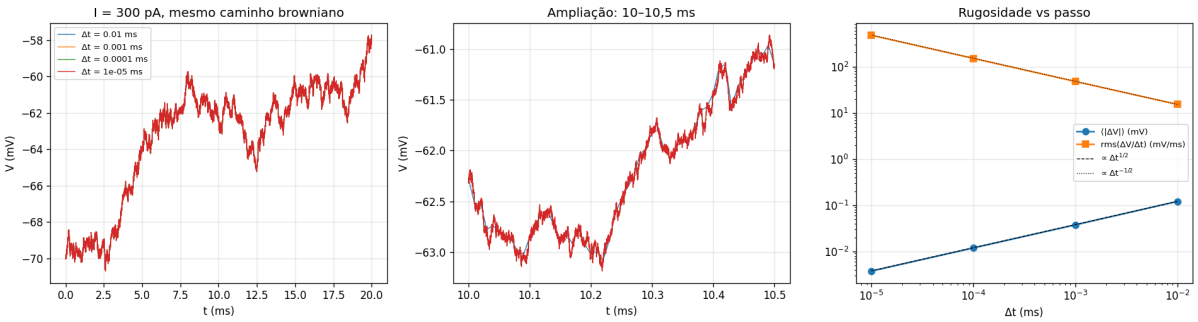



Figure 3: Trajectories for decreasing $\Delta t$ (left), magnified (center), and roughness measures (right).

| $\Delta t$ (ms) | $\langle\lvert\Delta V\rvert\rangle$ (mV) | $\sum(\Delta V)^2$ (mV$^2$) | rms$(\Delta V/\Delta t)$ (mV/ms) |
|---|---|---|---|
| $10^{-2}$ | 0.1213 | 46.15 | 15.2 |
| $10^{-3}$ | 0.0378 | 45.11 | 47.5 |
| $10^{-4}$ | 0.0120 | 44.95 | 149.9 |
| $10^{-5}$ | 0.0038 | 44.90 | 473.8 |

**Conclusion.** The increment per step scales as $\sqrt{\Delta t}$ (it decreases by $\sqrt{10}$ for each decade), so the discrete path converges to a *continuous* curve: jumps vanish as $\Delta t \to 0$ and the trajectories for different $\Delta t$ overlap at the scale of the figure. However, the path does not become differentiable: the slope $\Delta V/\Delta t \propto \Delta t^{-1/2}$ diverges, and the roughness remains at every magnification (center panel). Consistently, the quadratic variation $\sum(\Delta V)^2 \to \sigma^2 t = 45$ mV$^2$ is independent of $\Delta t$. Hence "smooth" means continuous, as in Brownian motion, not differentiable; this is also why $\sigma\tilde{w}_n\sqrt{\Delta t}$ (and not $\Delta t$) is the correct scaling of the noise term.

## 2.3 (c) Regular and irregular firing

We use $\sigma = 1.5$ mV/ms$^{1/2}$ and, because the suggested values do not give a good contrast for this $\sigma$, $I_{\text{below}} = 400$ pA and $I_{\text{above}} = 800$ pA ($I_L = 500$ pA). Figure 4 shows (top) $V(t)$ with the spike mechanism turned off and the threshold (dashed) and (bottom) the same noise realization with spikes (peaks drawn by hand at +20 mV); the first 1000 ms of the 2 s run are shown.

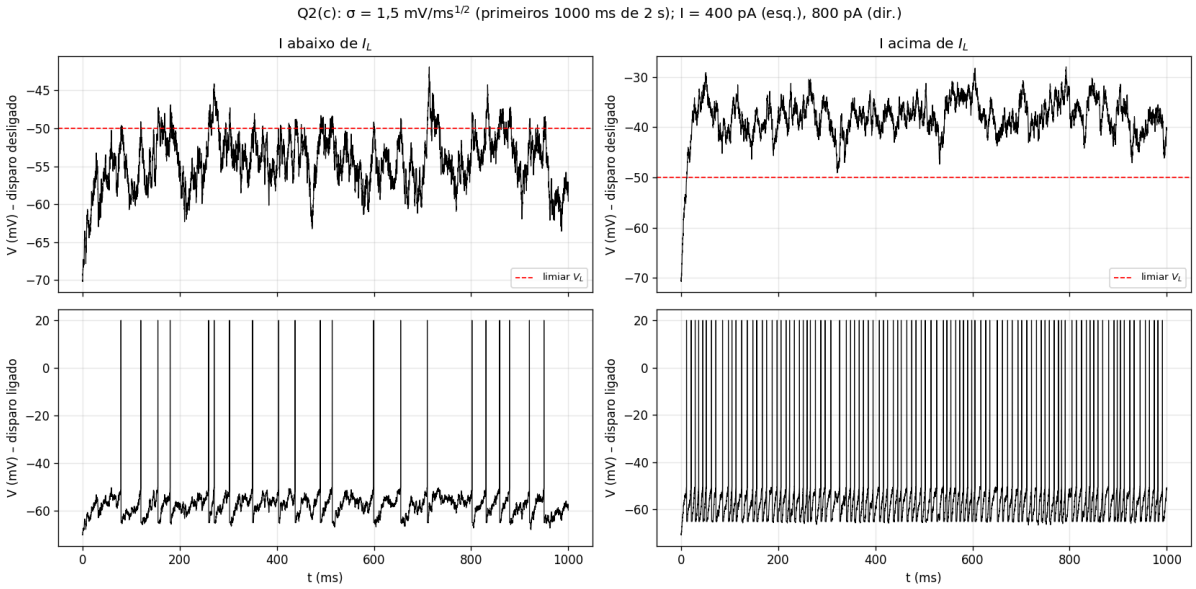



Figure 4: Below threshold (left) and above threshold (right).

| Case | $I$ (pA) | $N$ | $\langle$ISI$\rangle$ (ms) | sd (ms) | $CV_{\text{ISI}}$ |
|---|---|---|---|---|---|
| Below | 400 | 40 | 48.56 | 27.26 | 0.56 |
| Above | 800 | 203 | 9.83 | 2.67 | 0.27 |

Averaged over 30 realizations: $CV = 0.67 \pm 0.11$ at 400 pA and $0.23 \pm 0.014$ at 800 pA; the CV decreases monotonically with $I$ (Fig. 5). The firing is more irregular below threshold. There, $V_\infty < V_L$ and spikes are triggered by rare noise excursions to the threshold; the waiting times are broadly distributed (close to the exponential of a Poisson process, though $CV < 1$ because of the refractory period and the reset), so the ISI distribution is wide. Above threshold the mean drive makes $V$ climb deterministically toward $V_L$ and noise only perturbs this ramp, so ISIs cluster around the deterministic charging time (plus $\tau_{\text{ref}}$): firing is regular.

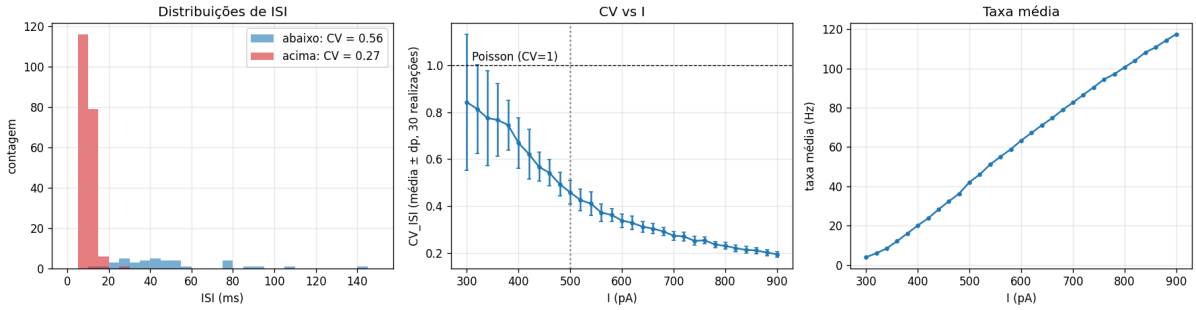



Figure 5: ISI histograms, $CV_{\text{ISI}}$ vs. $I$ and mean rate (30 seeds per current).

# 3 Question 3: LIF with adaptation current

The model is $C\dot{V} = (V_{\text{rep}} - V)/R + G_a(V_{\text{redef}} - V) + I$, $\dot{G}_a = -G_a/\tau_a$, with $V \to V_{\text{redef}}$ and $G_a \to G_a + b$ at each spike; $V_{\text{redef}} = -80$ mV, $b = 1$ nS, $\tau_a = 200$ ms, other parameters as in Question 1 (including $\tau_{\text{ref}}$).

## 3.1 (a) Current pulse

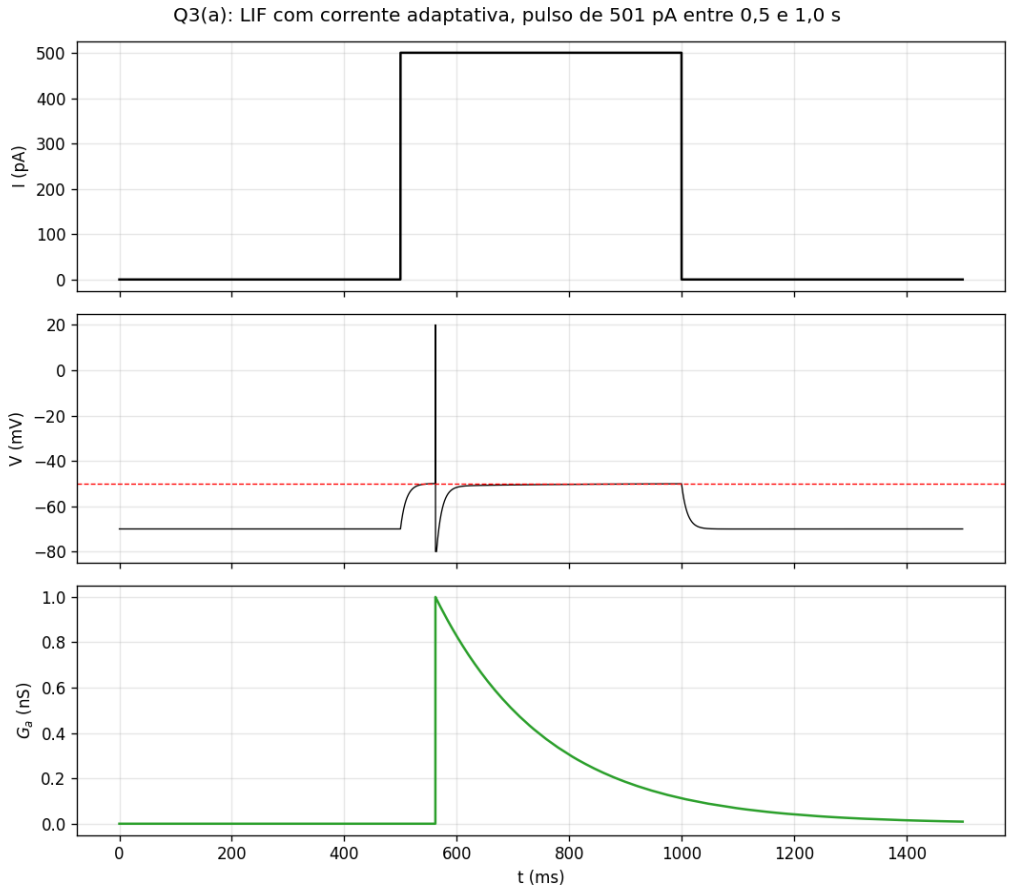



Figure 6: Applied current, membrane potential and adaptation conductance for a 501 pA pulse between 0.5 and 1.0 s.

Only **one spike** occurs, 62.13 ms after the pulse onset, in agreement with the analytical latency $\tau_m \ln \frac{RI}{RI-20} = 10 \ln 501 = 62.17$ ms (with $G_a = 0$). Afterwards $G_a = 1$ nS and decays with $\tau_a$. A new spike requires $V_\infty^{\text{eff}} = \frac{G_L V_{\text{rep}} + G_a V_{\text{redef}} + I}{G_L + G_a} \geq V_L$ ($G_L = 1/R = 25$ nS), i.e.

$$G_a \leq \frac{I - I_L}{V_L - V_{\text{redef}}} = \frac{501 - 500}{30} = \frac{1}{30} \text{ nS}. \tag{3}$$

This needs $t - t_1 = \tau_a \ln 30 = 680$ ms, longer than the 438 ms left in the pulse (at the end of the pulse $G_a = e^{-438/200} = 0.11$ nS), so the neuron stays silent; $G_a$ keeps decaying after the pulse.

## 3.2 (b) $f_1$–$I$ and $f_\infty$–$I$ curves

Simulations of 5 s for $400 \le I \le 800$ pA (20 pA steps); $T_1 = t_2 - t_1$ and $T_\infty$ is the last ISI (after about $25\tau_a$), $f_1 = 1/T_1$, $f_\infty = 1/T_\infty$.

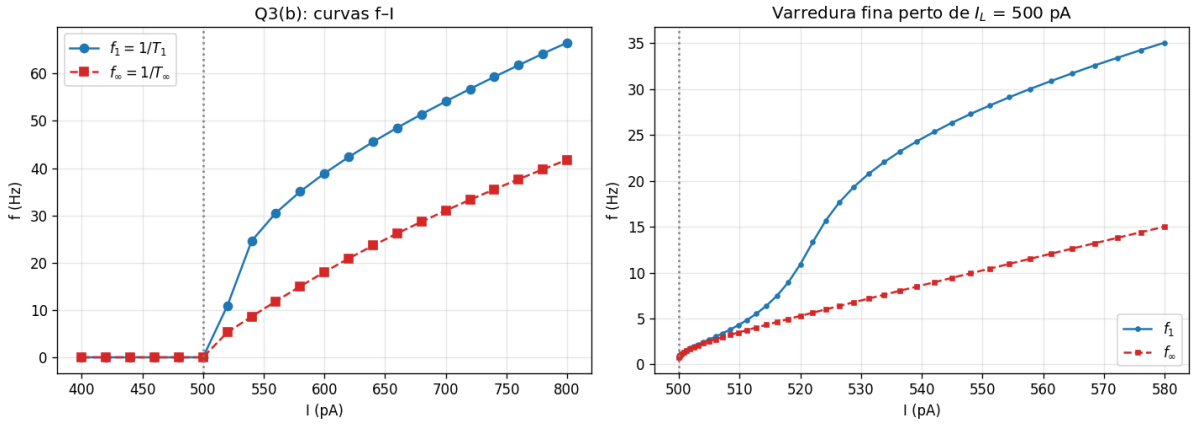



Figure 7: Left: $f_1$ and $f_\infty$ vs. $I$. Right: fine sweep near $I_L = 500$ pA.

| $I$ (pA) | 500 | 520 | 600 | 700 | 800 |
|---|---|---|---|---|---|
| $f_1$ (Hz) | 0 | 10.9 | 38.9 | 54.1 | 66.5 |
| $f_\infty$ (Hz) | 0 | 5.3 | 18.0 | 31.0 | 41.8 |

Adaptation lowers the steady rate, $f_\infty < f_1$ (about 60–65% of $f_1$ at the highest currents). Both curves start at $I_L = 500$ pA, where no spikes are produced (the response disappears with $I \le I_L$ even with $G_a = 0$), and the fine sweep shows $f_1$ and $f_\infty$ decreasing continuously to $\approx 0$ as $I \to I_L^+$ ($f \approx 0.8$ Hz, i.e. $T \approx 1.3$ s, already at 500.05 pA and $f = 0$ at 500.0 pA; periods longer than the 5 s window cannot be measured). The frequency therefore starts from zero: the model is **type I**, as the LIF neuron without adaptation (logarithmic divergence of the period at $I_L$).

# A Fortran code

## Common module

```
!  ======================================================================
!   neuro_mod : parametros e rotinas comuns da Lista 6 (LIF, ruido, adaptacao)
!   Unidades: tempo [ms], tensao [mV], corrente [pA], capacitancia [pF],
!             resistencia [MOhm], condutancia [nS].   (1 pA/pF = 1 mV/ms)
!  ======================================================================
module neuro_mod
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: Vrep = -70.0_dp, Rm = 40.0_dp, Cm = 250.0_dp
  real(dp), parameter :: VL = -50.0_dp, Vredef0 = -65.0_dp, tref = 2.0_dp
  real(dp), parameter :: taum = Rm*Cm*1.0e-3_dp          ! = 10 ms
  real(dp), parameter :: Vpico = 20.0_dp                 ! so para desenhar o disparo
  real(dp), parameter :: tbig = 1.0e30_dp
  logical,  private   :: tem_spare = .false.
  real(dp), private   :: spare = 0.0_dp
contains
  pure function RI(I)                        ! R*I em mV (I em pA, R em MOhm)
    real(dp), intent(in) :: I
    real(dp) :: RI
```

```
    RI = Rm*I*1.0e-3_dp
  end function
  ! f-I analitico do LIF com periodo refratario absoluto (Hz)
  pure function f_analitica(I, Vredef) result(f)
    real(dp), intent(in) :: I, Vredef
    real(dp) :: f, x
    x = RI(I)
    if (x <= VL - Vrep) then
      f = 0.0_dp
    else
      f = 1000.0_dp/(tref + taum*log((x - (Vredef - Vrep))/(x - (VL - Vrep))))
    end if
  end function
  subroutine set_seed(k)                    ! semente diferente para cada simulacao
    integer, intent(in) :: k
    integer :: n, i
    integer, allocatable :: seed(:)
    call random_seed(size=n)
    allocate(seed(n))
    do i = 1, n
      seed(i) = int(mod(int(k,8)*2654435761_8 + int(i,8)*40503_8*7919_8 + 12345_8,
          2147483647_8))
      if (seed(i) == 0) seed(i) = 1
    end do
    call random_seed(put=seed)
    tem_spare = .false.
  end subroutine
  function gauss() result(g)                ! N(0,1) por Box-Muller
    real(dp) :: g, u1, u2, r
    if (tem_spare) then
      g = spare; tem_spare = .false.; return
    end if
    call random_number(u1); call random_number(u2)
    if (u1 < 1.0e-300_dp) u1 = 1.0e-300_dp
    r = sqrt(-2.0_dp*log(u1))
    g = r*cos(2.0_dp*acos(-1.0_dp)*u2)
    spare = r*sin(2.0_dp*acos(-1.0_dp)*u2); tem_spare = .true.
  end function
  !-----------------------------------------------------------------------
  !  Integracao do LIF (Euler / Euler-Maruyama) com:
  !    * corrente I aplicada em ton <= t < toff (constante: ton=-1, toff=tbig)
  !    * ruido sigma [mV/ms^1/2]              (sigma = 0  -> deterministico)
  !    * adaptacao (b [nS], taua [ms]); b = 0 desliga
  !    * free=.true. : mecanismo de disparo desligado (sem limiar/refratario)
  !  Disparo: V_{k+1} >= VL e V_k < VL ; t* por interpolacao linear (Fig. 1).
  !  O gaussiano e' sorteado em TODO passo (mesmo no refratario), assim a
  !  trajetoria com e sem disparos usa exatamente o mesmo ruido.
  !-----------------------------------------------------------------------
  subroutine run_lif(I, ton, toff, sigma, Vredef, dt, tmax, b, taua, spk, nspk, &
                     stride, Vtr, Gtr, ntr, free)
    real(dp), intent(in)  :: I, ton, toff, sigma, Vredef, dt, tmax, b, taua
    real(dp), intent(out) :: spk(:)
    integer,  intent(out) :: nspk
    integer,  intent(in),  optional :: stride
    real(dp), intent(out), optional :: Vtr(:), Gtr(:)
    integer,  intent(out), optional :: ntr
    logical,  intent(in),  optional :: free
    integer  :: n, nsteps, st, m
    real(dp) :: t, V, Vn, G, Gn, Iap, xi, refend, tstar, sq
    logical  :: fr, rec, disparou, pend
    fr = .false.; if (present(free)) fr = free
    rec = present(Vtr)
    st = 1; if (present(stride)) st = stride
    nsteps = nint(tmax/dt); sq = sqrt(dt)
    V = Vrep; G = 0.0_dp; refend = -1.0_dp; nspk = 0; m = 0; pend = .false.
    if (rec) then
      Vtr(1) = V; if (present(Gtr)) Gtr(1) = G; m = 1
    end if
    do n = 0, nsteps-1
      t = n*dt
      xi = 0.0_dp; if (sigma > 0.0_dp) xi = gauss()
      Iap = 0.0_dp; if (t >= ton .and. t < toff) Iap = I
      disparou = .false.
```

```
      if (.not. fr .and. t < refend) then            ! periodo refratario absoluto
        Vn = Vredef
        Gn = G; if (taua > 0.0_dp) Gn = G - dt*G/taua
      else
        Vn = V + dt*((Vrep - V)/taum + G*(Vredef - V)/Cm + Iap/Cm) + sigma*sq*xi
        Gn = G; if (taua > 0.0_dp) Gn = G - dt*G/taua
        if (.not. fr .and. Vn >= VL .and. V < VL) then
          tstar = t + dt*(VL - V)/(Vn - V)
          nspk = nspk + 1
          if (nspk <= size(spk)) spk(nspk) = tstar
          refend = tstar + tref
          disparou = .true.
        end if
      end if
      V = Vn; G = Gn
      if (disparou) then                              ! disparo "posto a mao"
        pend = .true.; V = Vredef; G = G + b
      end if
      if (rec .and. mod(n+1, st) == 0) then
        m = m + 1
        if (pend) then
          Vtr(m) = Vpico; pend = .false.             ! pico aparece na amostra seguinte
              ao disparo
        else
          Vtr(m) = V
        end if
        if (present(Gtr)) Gtr(m) = G
      end if
    end do
    if (present(ntr)) ntr = m
  end subroutine
  subroutine isi_stats(spk, nspk, mu, sd, cv)
    real(dp), intent(in)  :: spk(:)
    integer,  intent(in)  :: nspk
    real(dp), intent(out) :: mu, sd, cv
    real(dp), allocatable :: isi(:)
    integer :: k
    mu = 0.0_dp; sd = 0.0_dp; cv = 0.0_dp
    if (nspk < 3) return
    allocate(isi(nspk-1))
    do k = 1, nspk-1
      isi(k) = spk(k+1) - spk(k)
    end do
    mu = sum(isi)/size(isi)
    sd = sqrt(sum((isi-mu)**2)/(size(isi)-1))       ! desvio padrao amostral
    cv = sd/mu
  end subroutine
end module neuro_mod
```

# Q1(a)

```
! Questao 1 (a): curva f-I do LIF com periodo refratario; numerica x analitica
!   Saida: data/q1/q1a_fI.dat  (I[pA]  f_num[Hz]  f_analitica[Hz])
program q1a_fI_lif
  use neuro_mod
  implicit none
  real(dp), parameter :: dt = 0.01_dp, tmax = 2000.0_dp
  real(dp) :: I, spk(5000)
  integer :: nspk, k
  open(10, file='data/q1/q1a_fI.dat', status='replace')
  write(10,'(A)') '# I[pA]   f_num[Hz]   f_analitica[Hz]     (IL = 500 pA)'
  do k = 0, 120
    I = 400.0_dp + 10.0_dp*k                          ! 400 ... 1600 pA
    call run_lif(I, -1.0_dp, tbig, 0.0_dp, Vredef0, dt, tmax, 0.0_dp, 0.0_dp, spk, nspk)
    write(10,'(F8.1,2(1x,F12.5))') I, nspk*1000.0_dp/tmax, f_analitica(I, Vredef0)
  end do
  close(10)
  print *, 'q1a ok'
end program
```

## Q2(a)

```
! Questao 2 (a): f-I do LIF com ruido (Euler-Maruyama). Uso: q2a_fI_ruido sig1 sig2 ...
!   Saida: data/q2/q2a_fI_sigma_<sig>.dat   (I  f[Hz])    sigma em mV/ms^(1/2)
program q2a_fI_ruido
  use neuro_mod
  implicit none
  real(dp), parameter :: dt = 0.01_dp, tmax = 2000.0_dp
  real(dp) :: I, sigma, spk(20000)
  integer :: nspk, k, ia, na
  character(len=32) :: arg
  na = command_argument_count()
  do ia = 1, na
    call get_command_argument(ia, arg); read(arg,*) sigma
    open(10, file='data/q2/q2a_fI_sigma_'//trim(arg)//'.dat', status='replace')
    write(10,'(A,F8.3)') '# I[pA]   f[Hz]   sigma = ', sigma
    do k = 0, 130
      I = 300.0_dp + 10.0_dp*k                    ! 300 ... 1600 pA
      call set_seed(100000*ia + k + 1)            ! sequencia diferente por corrente
      call run_lif(I, -1.0_dp, tbig, sigma, Vredef0, dt, tmax, 0.0_dp, 0.0_dp, spk, nspk)
      write(10,'(F8.1,1x,F12.5)') I, nspk*1000.0_dp/tmax
    end do
    close(10)
  end do
  print *, 'q2a ok'
end program
```

## Q2(b)

```
! Questao 2 (b): rugosidade da trajetoria subliminar vs passo de tempo.
! O MESMO caminho browniano (ruido sorteado no passo mais fino) e' agregado
! para os passos maiores, de modo que as trajetorias sao diretamente comparaveis.
! Uso: q2b_dt_rugosidade sigma [I]        (I = 300 pA por padrao)
program q2b_dt_rugosidade
  use neuro_mod
  implicit none
  integer, parameter :: nlev = 5
  real(dp), parameter :: dtf = 1.0e-5_dp, tmax = 20.0_dp
  real(dp) :: dts(nlev), sigma, I, V, dt, dW, t, dV, sabs, s2, sd2, vold
  real(dp), allocatable :: xi(:)
  integer :: nf, lev, m, n, nsteps, k, strideo
  character(len=64) :: arg, fn
  call get_command_argument(1, arg); read(arg,*) sigma
  I = 300.0_dp
  if (command_argument_count() > 1) then
    call get_command_argument(2, arg); read(arg,*) I
  end if
  dts = (/ 1.0e-2_dp, 1.0e-3_dp, 1.0e-4_dp, 1.0e-5_dp, 1.0e-5_dp /)
  nf = nint(tmax/dtf)
  allocate(xi(nf)); call set_seed(777)
  do k = 1, nf
    xi(k) = gauss()
  end do
  open(20, file='data/q2/q2b_metricas.dat', status='replace')
  write(20,'(A)') '# dt[ms]   mean|dV|[mV]   rms(dV)[mV]   QV=sum(dV^2)[mV^2]   rms(dV/dt)[mV
      /ms]   sigma^2*T'
  do lev = 1, nlev-1
    dt = dts(lev); m = nint(dt/dtf); nsteps = nint(tmax/dt)
    write(fn,'(A,I0,A)') 'data/q2/q2b_traj_dt', lev, '.dat'
    open(21, file=trim(fn), status='replace')
    write(fn,'(A,I0,A)') 'data/q2/q2b_zoom_dt', lev, '.dat'
    open(22, file=trim(fn), status='replace')
    write(21,'(A,ES10.2)') '# t[ms] V[mV]   dt = ', dt
    write(22,'(A,ES10.2)') '# t[ms] V[mV]   dt = ', dt
    strideo = max(1, nint(1.0e-4_dp/dt))
    V = Vrep; sabs = 0; s2 = 0; sd2 = 0
    write(21,'(2(1x,ES14.6))') 0.0_dp, V
    do n = 0, nsteps-1
      t = n*dt
      dW = sqrt(dtf)*sum(xi(n*m+1:(n+1)*m))
```

```
      vold = V
      V = V + dt*((Vrep - V)/taum + I/Cm) + sigma*dW
      dV = V - vold
      sabs = sabs + abs(dV); s2 = s2 + dV**2; sd2 = sd2 + (dV/dt)**2
      if (mod(n+1, strideo) == 0) write(21,'(2(1x,ES14.6))') (n+1)*dt, V
      if ((n+1)*dt >= 10.0_dp .and. (n+1)*dt <= 10.5_dp) write(22,'(2(1x,ES14.6))') (n+1)
          *dt, V
    end do
    write(20,'(ES10.2,4(1x,ES14.6),1x,ES14.6)') dt, sabs/nsteps, sqrt(s2/nsteps), s2,
        sqrt(sd2/nsteps), sigma**2*tmax
    close(21); close(22)
  end do
  close(20)
  print *, 'q2b ok'
end program
```

# Q2(c)

```
! Questao 2 (c): regime regular x irregular. Uso: q2c_cv_isi sigma Iabaixo Iacima
!   data/q2/q2c_trace_{abaixo,acima}.dat : t  V_sem_disparo  V_com_disparo
!   data/q2/q2c_spikes_{abaixo,acima}.dat: instantes dos disparos e ISIs
!   data/q2/q2c_stats.dat                : <ISI>, sd(ISI), CV (uma realizacao)
!   data/q2/q2c_cv_ensemble.dat          : CV medio sobre 100 sementes
program q2c_cv_isi
  use neuro_mod
  implicit none
  real(dp), parameter :: dt = 0.01_dp, tmax = 2000.0_dp
  integer, parameter :: stride = 10, nmax = int(tmax/dt)/stride + 2
  real(dp) :: sigma, Is(2), spk(20000), mu, sd, cv, I
  real(dp), allocatable :: Vf(:), Vo(:)
  integer :: nspk, nt, nt2, ic, k, js, nr
  real(dp) :: cvs(100), rates(100), cm_, cs_
  character(len=16) :: lab(2), arg
  character(len=64) :: fn
  call get_command_argument(1, arg); read(arg,*) sigma
  call get_command_argument(2, arg); read(arg,*) Is(1)
  call get_command_argument(3, arg); read(arg,*) Is(2)
  lab = (/ 'abaixo', 'acima ' /)
  allocate(Vf(nmax), Vo(nmax))
  open(30, file='data/q2/q2c_stats.dat', status='replace')
  write(30,'(A,F7.3)') '# sigma = ', sigma
  write(30,'(A)') '# caso   I[pA]   Nspk   <ISI>[ms]   sd(ISI)[ms]   CV'
  do ic = 1, 2
    call set_seed(5000 + ic)
    call run_lif(Is(ic), -1.0_dp, tbig, sigma, Vredef0, dt, tmax, 0.0_dp, 0.0_dp, spk,
        nspk, stride, Vo, ntr=nt)
    call set_seed(5000 + ic)                          ! mesmo ruido, disparos desligados
    call run_lif(Is(ic), -1.0_dp, tbig, sigma, Vredef0, dt, tmax, 0.0_dp, 0.0_dp, spk
        (1:1), nr, stride, Vf, ntr=nt2, free=.true.)
    call set_seed(5000 + ic)
    call run_lif(Is(ic), -1.0_dp, tbig, sigma, Vredef0, dt, tmax, 0.0_dp, 0.0_dp, spk,
        nspk)
    call isi_stats(spk, nspk, mu, sd, cv)
    write(30,'(A8,F8.1,I6,3(1x,F10.4))') trim(lab(ic)), Is(ic), nspk, mu, sd, cv
    fn = 'data/q2/q2c_trace_'//trim(lab(ic))//'.dat'
    open(31, file=trim(fn), status='replace')
    write(31,'(A)') '# t[ms]   V_sem_disparo[mV]   V_com_disparo[mV]'
    do k = 1, nt
      write(31,'(F10.3,2(1x,F10.4))') (k-1)*stride*dt, Vf(k), Vo(k)
    end do
    close(31)
    fn = 'data/q2/q2c_spikes_'//trim(lab(ic))//'.dat'
    open(31, file=trim(fn), status='replace')
    write(31,'(A)') '# k   t_k[ms]   ISI_k[ms]   (ultimo disparo: ISI = 0)'
    do k = 1, nspk
      if (k < nspk) then
        write(31,'(I5,2(1x,F12.4))') k, spk(k), spk(k+1)-spk(k)
      else
        write(31,'(I5,2(1x,F12.4))') k, spk(k), 0.0_dp
      end if
```

```
    end do
    close(31)
  end do
  close(30)
  ! ensemble: CV e taxa vs I (30 sementes por corrente), mesmo sigma
  open(32, file='data/q2/q2c_cv_ensemble.dat', status='replace')
  write(32,'(A,F7.3)') '# I[pA]   <CV>   sd(CV)   <rate>[Hz]      sigma = ', sigma
  do k = 0, 30
    I = 300.0_dp + 20.0_dp*k
    nr = 0
    do js = 1, 30
      call set_seed(900000 + 100*k + js)
      call run_lif(I, -1.0_dp, tbig, sigma, Vredef0, dt, tmax, 0.0_dp, 0.0_dp, spk, nspk)
      call isi_stats(spk, nspk, mu, sd, cv)
      if (nspk >= 3) then
        nr = nr + 1; cvs(nr) = cv
      end if
      rates(js) = nspk*1000.0_dp/tmax
    end do
    if (nr > 1) then
      cm_ = sum(cvs(1:nr))/nr; cs_ = sqrt(sum((cvs(1:nr)-cm_)**2)/(nr-1))
    else
      cm_ = 0; cs_ = 0
    end if
    write(32,'(F8.1,3(1x,F10.4))') I, cm_, cs_, sum(rates(1:30))/30
  end do
  close(32)
  print *, 'q2c ok'
end program
```

# Q3(a)

```
! Questao 3 (a): LIF com corrente adaptativa; pulso I = 501 pA entre 0,5 e 1,0 s
!   data/q3/q3a_trace.dat (t[ms] I[pA] V[mV] Ga[nS]) ; data/q3/q3a_spikes.dat
program q3a_pulso_adaptacao
  use neuro_mod
  implicit none
  real(dp), parameter :: dt = 0.01_dp, tmax = 1500.0_dp, Ip = 501.0_dp
  real(dp), parameter :: Vredef = -80.0_dp, b = 1.0_dp, taua = 200.0_dp
  integer, parameter :: stride = 10, nmax = int(tmax/dt)/stride + 2
  real(dp) :: spk(2000), t
  real(dp), allocatable :: V(:), G(:)
  integer :: nspk, nt, k
  allocate(V(nmax), G(nmax))
  call run_lif(Ip, 500.0_dp, 1000.0_dp, 0.0_dp, Vredef, dt, tmax, b, taua, spk, nspk,
      stride, V, G, nt)
  open(10, file='data/q3/q3a_trace.dat', status='replace')
  write(10,'(A)') '# t[ms]   I[pA]   V[mV]   Ga[nS]'
  do k = 1, nt
    t = (k-1)*stride*dt
    write(10,'(F10.3,F8.1,2(1x,F12.5))') t, merge(Ip, 0.0_dp, t >= 500.0_dp .and. t <
        1000.0_dp), V(k), G(k)
  end do
  close(10)
  open(11, file='data/q3/q3a_spikes.dat', status='replace')
  write(11,'(A)') '# k   t_k[ms]   ISI_k[ms]   (ultimo disparo: ISI = 0)'
  do k = 1, nspk
    if (k < nspk) then
      write(11,'(I4,2(1x,F12.4))') k, spk(k), spk(k+1)-spk(k)
    else
      write(11,'(I4,2(1x,F12.4))') k, spk(k), 0.0_dp
    end if
  end do
  close(11)
  print *, 'q3a ok: ', nspk, ' disparos; t1 = ', spk(1) - 500.0_dp, ' ms apos o inicio do
      pulso'
end program
```

# Q3(b)

```
! Questao 3 (b): f1 = 1/T1 e f_inf = 1/T_inf vs I (400..800 pA, passo 20 pA), T = 5 s
!   data/q3/q3b_fI.dat      : I  Nspk  T1  Tinf  f1  finf      (T em ms, f em Hz)
!   data/q3/q3b_fI_fino.dat : varredura fina perto de IL = 500 pA (so f1)
program q3b_fI_adaptacao
  use neuro_mod
  implicit none
  real(dp), parameter :: dt = 0.01_dp, tmax = 5000.0_dp, Vredef = -80.0_dp
  real(dp), parameter :: b = 1.0_dp, taua = 200.0_dp
  real(dp) :: I, spk(5000), T1, Tinf, f1, finf
  integer :: nspk, k
  open(10, file='data/q3/q3b_fI.dat', status='replace')
  write(10,'(A)') '# I[pA]   Nspk   T1[ms]   Tinf[ms]   f1[Hz]   finf[Hz]    (0 = indefinido)'
  do k = 0, 20
    I = 400.0_dp + 20.0_dp*k
    call medidas(I)
    write(10,'(F8.1,I6,4(1x,F12.4))') I, nspk, T1, Tinf, f1, finf
  end do
  close(10)
  open(11, file='data/q3/q3b_fI_fino.dat', status='replace')
  write(11,'(A)') '# I[pA]   Nspk   T1[ms]   Tinf[ms]   f1[Hz]   finf[Hz]'
  do k = 0, 40
    I = 500.0_dp + 0.05_dp*k**2 /1.0_dp              ! 500 ... 580 pA, mais denso perto de
        IL
    call medidas(I)
    write(11,'(F9.3,I6,4(1x,F12.4))') I, nspk, T1, Tinf, f1, finf
  end do
  close(11)
  print *, 'q3b ok'
contains
  subroutine medidas(I)
    real(dp), intent(in) :: I
    call run_lif(I, -1.0_dp, tbig, 0.0_dp, Vredef, dt, tmax, b, taua, spk, nspk)
    T1 = 0; Tinf = 0; f1 = 0; finf = 0
    if (nspk >= 2) then
      T1 = spk(2) - spk(1); f1 = 1000.0_dp/T1
    end if
    if (nspk >= 3) then
      Tinf = spk(nspk) - spk(nspk-1); finf = 1000.0_dp/Tinf    ! ultimo ISI (regime
          estacionario)
    end if
  end subroutine
end program
```